



MAA × SHOPLINE — App Architecture & Resilient Inference Plan

Bottom line

The app is a SHOPLINE-scoped fork of MAA OS. It covers the onboarding and intelligence screens plus the 4 pilot features, all behind one inference gateway. Every model role has a ranked chain of providers. A request fails over on errors, timeouts, quota exhaustion and failed output checks, not just on errors.

What the design guarantees:

- The merchant never sees an error screen or a raw provider failure.
- The merchant never receives output that failed the schema, language, safety or quality checks.
- When no admitted model can meet the bar in time, the job is queued or served from the last valid result, clearly dated. It is never silently downgraded.

What it cannot guarantee: identical quality from free backups. A backup joins a chain only after it passes a side-by-side eval against that role’s primary. Where no backup passes (vision, image generation, Arabic customer copy), the chain queues instead of degrading.

Three measured facts from the 26 Sep architecture doc shape the rest of this plan:

- Nemotron Super returns HTTP 503 on roughly 2 of 5 calls, so it cannot be the only path for any interactive screen.
- The OpenRouter tier returns HTTP 402 (unfunded), so there is no working third tier today.
- The judge runs in shadow mode with an uncalibrated 0.35 threshold. Until it is calibrated, only error-based failover can be enforced, not quality-based failover.

Isolation: fork the code, share no runtime

The app reuses MAA OS code, such as `llm_router`, the retirement chain and `judge_is_independent()`, as a copy that lives in its own repo. It shares no running service, database, storage bucket, key or deployment with the main platform. A change to MAA OS reaches the SHOPLINE app only as a reviewed merge into the fork.

Shared	Not shared
Source code, copied into the SHOPLINE repo	Supabase project, buckets, RPCs, Edge Functions



Shared	Not shared
Design patterns and the governed pipeline shape	Provider API keys and quotas
The SKB snapshot, exported as a versioned file	Logs, trace rows, job queue, cache

Two guards enforce this. `isolation_check.py` fails CI on any main-platform identifier or foreign Supabase ref, and a startup assertion refuses to boot in production if any endpoint points outside the SHOPLINE project. The allowed project ref still has to be confirmed, because one source spells it `jojyjjpibkyafbpuungg` and another spells it `j0jyjjpibkyafbpuungg`.

Partnership scope and data classes

The app includes only what the signed Development and Referral Agreement (6 Aug 2026) and the proposal’s pilot cover. Everything else in MAA OS stays out of this build.

MAA OS concept	In the SHOPLINE app	Basis
Brand DNA, Audience, SWOT onboarding	Yes, as the calibration flow	Needed to personalise the pilot features
Ecosystem Connect	SHOPLINE store data; socials only with merchant consent	§7.2 explicit consent, §7.3 minimum data
Marketing Health Check	Yes	Proposal pilot activity
Voice of Customer	Yes, from reviews and feedback the merchant permits	Proposal pilot, subject to merchant permission
AI Growth Advisor	Yes	Proposal pilot and integration phase
AI Campaign Planner	Yes	Proposal pilot and integration phase
Strategy and Opportunity engines	Back-end engines that feed the four features, not separate products	Supports the pilot features
Shopify / WooCommerce connectors	No	§5.3c: no steering merchants to competing platforms
CRM sync, WhatsApp inbox, autonomous budget moves	Not in pilot scope	Add only by written amendment
Neural OS API over merchant data	No	§8.2 confidentiality, §7.3 purpose limit
Per-brand fine-tuning on merchant data	Only with separate explicit consent	§7.3 purpose limit
SHOPLINE name and logo in the UI	Only after written consent	§6.3

Three contract terms decide how models are routed:

- **Merchant Data is Confidential Information (§8.1).** It may only go to parties who need it to deliver the app. A provider that logs, retains or trains on prompts cannot receive it.
- **Cross-border transfer rules apply (§7.4).** A prompt sent to US-hosted inference is a transfer and needs a lawful basis in the privacy policy.



- **Breach notice is due within 2 business days (\$7.5).** A prompt misrouted to a non-approved provider counts as a suspected breach, so the router logs the destination of every call.

So every call carries a data class, either merchant-confidential or public. Merchant-confidential calls may only use providers on the approved list.

Architecture

The app uses the same governed shape as MAA OS: deterministic work first, one generative step, then validation before delivery. SHOPLINE webhooks never write directly to the data layer; they pass a verifier first.



One gateway for every model call, one verifier for every webhook



Every call writes a trace row: role, data class, provider, model, trigger, latency

App architecture · 7 layers

Only the inference gateway may call a model provider. Only verified webhook events reach merchant data, and only validated output reaches the merchant or the cache.

Layers and screen mapping

Each layer has one rule it never breaks.

Layer	Responsibility	Key rule
Client	Onboarding calibration, Home, the 4 pilot features, EN/AR RTL	Shows NOT COMPUTED or a dated result, never an error
SHOPLINE integration	OAuth install, minimum scopes, verified webhooks for orders, products and reviews	Tag every record with merchant and referral source at ingestion



MARKETING AS ART

BUILT ON SCIENCE. DRIVEN BY INTELLIGENCE. ENGINEERED FOR OUTCOMES.

MAA AI OS
APP ARCHITECTURE & RESILIENCE PLAN

Layer	Responsibility	Key rule
Merchant data	Brand DNA, personas, store snapshot, consent ledger, revenue ledger	RLS per merchant; consent checked before any read
Governed pipeline	Steps 1–9 of the 11-step contract, coverage gate at 0.70	Below the gate, no model is called
Inference gateway	Picks the model per role, fails over, enforces data class	The only code allowed to call a provider
Validation	Schema, language, CoT leak, safety, judge	Nothing reaches the merchant without passing
Queue and cache	Async jobs, last valid results, nightly precompute	Absorbs outages and quota exhaustion

Each screen depends on these roles:

Screen	Engines and roles used	Interactive or async
Brand DNA, Audience	Reasoning, Fast, Safety	Interactive
SWOT	Reasoning, Chain-of-thought analysis, Vision (optional)	Async with progress
Intelligence Initialization	All onboarding outputs, Embeddings	Async
Home	Cached intelligence, Fast	Interactive from cache
Marketing Health Check	Deterministic scoring, Reasoning, Judge	Nightly precompute; on-demand refresh is async
Voice of Customer	NLP (Gemini), Arabic, Reasoning	Nightly precompute
AI Growth Advisor	Reasoning (chat), Fast, Arabic, Safety	Interactive, streamed
AI Campaign Planner	Structured JSON, Content, Arabic copy, Safety, Judge, Image (optional)	Async with progress

Model roles and fallback chains

Each role has a primary and ranked backups. Model IDs are the ones the 26 Sep inventory lists as configured or measured. Rows marked *verify* need a live probe on the current keys before going into config. On the current Groq key only gpt-oss-20b and gpt-oss-120b are entitled; the Llama models return 404.

Role	Primary	Backup 1	Backup 2	Backup 3	If all fail
Reasoning, interactive	NVIDIA nemotron-3-super-120b-a12b, hedged	NVIDIA mistral-nemotron	Groq gpt-oss-120b	OpenRouter paid, zero-retention (<i>verify</i> ; needs credits)	Last valid result
Reasoning, batch (strategy, lead scoring)	Groq gpt-oss-120b	NVIDIA nemotron-3-super	NVIDIA mistral-nemotron	OpenRouter paid (<i>verify</i>)	Queue
Fast tier, automation logic	NVIDIA mistral-nemotron	Groq gpt-oss-20b	NVIDIA nemotron-3-super	OpenRouter paid (<i>verify</i>)	Queue



MARKETING AS ART

BUILT ON SCIENCE. DRIVEN BY INTELLIGENCE. ENGINEERED FOR OUTCOMES.

MAA AI OS
APP ARCHITECTURE & RESILIENCE PLAN

Role	Primary	Backup 1	Backup 2	Backup 3	If all fail
Structured JSON (Campaign Planner)	Groq gpt-oss-20b (NVIDIA gpt-oss-20b timed out at 30s)	NVIDIA gpt-oss-20b	Groq gpt-oss-120b	NVIDIA nemotron-3-super with JSON repair	Queue
Chain-of-thought analysis (SWOT, gaps)	deepseek-v4-pro-0813	nemotron-3-super	Groq gpt-oss-120b	OpenRouter paid (verify)	Queue
Arabic and bilingual copy	z-ai/glm-5.3	mistral-nemotron	nemotron-3-super	Groq gpt-oss-120b, internal drafts only	Queue customer-facing copy
Judge (pre-delivery)	mistral-nemotron	Groq gpt-oss-20b	NVIDIA gpt-oss-20b	none	NOT COMPUTED; hold if gates are enforced
Content safety	nemotron-3.5-content-safety	none	none	none	Block and queue (fail closed)
Vision	llama-3.2-11b-vision-instruct (502 since 25 Sep)	Gemini 2.5 Flash	OpenRouter paid vision (verify)	none	NOT COMPUTED
Image generation	flux.1-kontext-dev	OpenRouter paid image (verify)	none	none	Queue; show brief without image
Voice in (ASR)	parakeet-1.1b-rnnt-multilingual-asr	Groq Whisper (verify)	none	none	Text input
Voice out (TTS)	maggie-tts-multilingual	none	none	none	Text output
NLP and embeddings	Gemini 2.5 Flash-Lite, gemini-embedding-2	Gemini 2.5 Flash	none	none	Retry later

Five rules apply to this table:

- **Batch reasoning starts on Groq.** With Nemotron Super failing on about 40% of calls, a batch job that starts there wastes an attempt 2 times in 5. Batch work moves back to NVIDIA once its measured failure rate stays under 10% for a week.
- **Groq gpt-oss-120b is the strongest free backup available.** It is retired on NVIDIA but live on Groq at 2.1s.
- **Free backups are not equivalent for Arabic.** Groq gpt-oss is fine for internal reasoning in Arabic, not for merchant-facing Arabic copy, which queues instead.
- **Safety never substitutes.** A missing safety verdict blocks delivery.
- **Vision has no free equivalent on these keys.** Gemini 2.5 Flash is the realistic backup because it is already live and contracted.

Router design

The router is the forked llm_router and retirement chain, extended. It picks a candidate, calls it within a time budget, validates the answer and moves on if anything fails.



Candidate filtering, per call

1. Start from the role's ranked chain.
2. Drop providers not approved for the call's data class.
3. Drop models whose circuit breaker is open.
4. Drop models whose remaining quota is below the reserve for this request class.
5. Drop judge candidates from the same family as the model that actually generated the answer. This runs after substitution, as `judge_is_independent()` already does.

Failover triggers

- HTTP 429, 5xx, 404, 410 or a connection error.
- Streaming calls: no first token within the stage budget (interactive about 4s, standard 20s, batch 60s).
- Non-streaming calls, such as Campaign Planner JSON: no complete response within the total budget (interactive 8s, standard 30s, batch 90s).
- `finish_reason` is length, or the output is empty or truncated.
- JSON fails to parse or fails the schema. One repair attempt on the same model, then move on.
- Wrong language, or reasoning text leaking into customer copy.
- Safety block. Regenerate once, then queue.
- Judge score below threshold. This stays in shadow mode until the judge is calibrated.

Hedging for interactive screens. The Growth Advisor chat and Home refresh fire the primary first. If no token arrives by that model's p90 latency, the first backup starts in parallel and the first valid answer wins. Hedging costs extra quota, so it applies only to interactive calls. The existing 0.6 / 0.4 time split between NVIDIA and the backup stays as the outer limit.

Quota budgeting

- One token bucket per provider and model, fed by the rate-limit headers each provider returns.
- A daily reserve, for example 30%, is kept for interactive calls. Batch jobs stop drawing from a bucket once it drops to the reserve.
- The router predicts exhaustion from the burn rate and shifts batch work to the next provider before the limit is hit.

Circuit breakers. A model's breaker opens after 3 consecutive failures, or when more than 40% of its last 50 calls failed. It is probed once every 60s and closes after 2 successes. Nemotron Super's breaker will trip often; that is intended.



Backup admission (what makes “no quality impact” testable)

- Build a golden set per role: about 50 real-shaped prompts in English and 50 in Arabic.
- Run the primary and each backup on the set. The judge compares answers pairwise, and a person spot-checks 10%.
- A backup joins the live chain only if it wins or ties on at least 90% of cases. Re-run weekly and after any provider change.

Logging. Every call writes a trace row: role, data class, provider, model, attempt number, the failover trigger, latency, tokens and validation result. This covers observability and the \$7.5 breach-evidence requirement.

Never-error policy

The merchant sees one of four states, never a failure. The router walks down this ladder only after the full chain is exhausted.

Step	What the merchant sees	When it applies
1. Live answer	Fresh result from any admitted model	Normal path
2. Last valid result	Previous result with its date, e.g. “Health Check from 25 Sep, refreshing”	Precomputed screens: Home, Health Check, VoC
3. Queued job	“Your campaign plan is being prepared, ready in about 5 minutes” plus a push notification	Campaign Planner, SWOT, anything async
4. Honest gap	NOT COMPUTED chip on the one affected panel; the rest of the screen works	Vision, image, voice, or a failed safety check

Three practices keep most requests from reaching step 2:

- **Precompute off-peak.** Health Check and VoC run nightly per merchant on spare quota, so opening those screens costs no model call.
- **Cache by content.** The key is merchant, role, input hash and SKB snapshot. Identical requests within the validity window return the cached answer.
- **Retry queued jobs with backoff.** Jobs retry across the whole chain at 1, 5 and 15 minutes. After 30 minutes they are flagged in /admin, not to the merchant.

This ladder matches the data-truthfulness contract: no fabricated numbers, and a dated or missing value is labelled rather than hidden.

Capacity and limits

Free tiers can carry the pilot: one Live Customer plus a handful of merchants. They cannot carry a public SHOPLINE listing. The figures below are approximate and have not been re-checked; confirm each against the provider’s current terms and account dashboards.



Provider	Approximate free limit	Production and data terms to confirm
NVIDIA API catalog (developer program)	Around 40 requests per minute per account	Licensed for development and testing; production needs NIM licensing or a paid endpoint
Groq free tier	Per-model caps on requests per minute, requests per day and tokens per day	No-training and retention terms, before any merchant data is sent
OpenRouter free models	About 20 requests per minute; 50 per day, or about 1,000 per day after buying credits	Many free providers log prompts; data-collection deny and zero-retention routing excludes most free models

A rough pilot budget: one merchant onboarding is about 15–25 model calls, and a day of normal use is about 20–40 calls plus the nightly precompute. At 5 merchants that stays well under 500 calls a day. Free tiers cover that as long as the router spreads load and hedges only interactive calls.

Run the pilot on free tiers with this router. Before public launch, fund a small paid, zero-retention third tier and move the primary to a production-licensed NVIDIA endpoint.

Blockers and build order

The first four items block the guarantees in the bottom line. The rest can run in parallel.

1. **Confirm the Supabase project ref** (jojyjj... vs j0jyjj...) and lock it into CI and the startup assertion.
2. **Fund OpenRouter or pick another paid third tier.** Until then, the third backup is configured but returns 402.
3. **Approve providers for merchant data.** Record each provider’s retention and training terms against \$7.3, \$7.4 and \$8. Only approved providers get merchant-confidential calls.
4. **Calibrate the judge.** Score an archive of real outputs, re-derive the threshold for mistral-nemotron, then switch quality failover from shadow to enforced.
5. **Probe entitlements.** Confirm Groq Whisper, the Gemini vision fallback and the OpenRouter models marked *verify*.
6. **Build the golden sets and run backup admission** for each role, in English and Arabic.
7. **Ship the router pieces:** data-class filter, quota buckets, breakers, hedging, first-token and total-duration budgets, the validation chain and extended trace rows.
8. **Ship the webhook verifier, queue, cache and nightly precompute** for Health Check and VoC.
9. **Fix the open code items from the 26 Sep doc.** The strategy and content engine chains still name the retired openai/gpt-oss-120b on NVIDIA, and vision.py still calls the 90B vision model.



10. **Pilot with the Live Customer** on the custom-app install, then submit for SHOPLINE review.

One input is still missing: the UI file is a bundled HTML prototype whose screen code did not come through the Drive reader. The screen list above comes from the proposal and the product-design skill, not from that file.